Name: Charmi Gaddale

Roll Number: 23071A0516

AIM: Evaluating Model Performance: Supervised Learning- Classification Supervised Learning- Regression Unsupervised Learning-Clustering (Use Lower Back Pain Symptoms dataset available at Kaggle (https://www.kaggle.com/sammy123/lower-back-pain- symptoms-dataset). The dataset spint.csv consists of 310 observations and 13 attributes). Also identify the cluster quality with purity and silhouette width

Dataset: Lower Back Pain Symptoms dataset

```
#Upload Dataset
from google.colab import files
uploaded = files.upload()
```

```
Choose files  Dataset_spine.csv
Dataset_spine.csv(text/csv) - 42534 bytes, last modified: 15/04/2026 - 100% done
Saving Dataset_spine.csv to Dataset_spine.csv
```

```
#Import Required Libraries
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.preprocessing import StandardScaler

from sklearn.metrics import accuracy_score
from sklearn.metrics import mean_squared_error
from sklearn.metrics import r2_score
from sklearn.metrics import silhouette_score

from sklearn.linear_model import LogisticRegression
from sklearn.linear_model import LinearRegression

from sklearn.cluster import KMeans

import matplotlib.pyplot as plt
import seaborn as sns
```

```
data = pd.read_csv("Dataset_spine.csv")
data.head()
```

	Col1	Col2	Col3	Col4	Col5	Col6	Col7	Col8	Col9	Co
0	63.027817	22.552586	39.609117	40.475232	98.672917	-0.254400	0.744503	12.5661	14.5386	15.30
1	39.056951	10.060991	25.015378	28.995960	114.405425	4.564259	0.415186	12.8874	17.5323	16.78
2	68.832021	22.218482	50.092194	46.613539	105.985135	-3.530317	0.474889	26.8343	17.4861	16.65
3	69.297008	24.652878	44.311238	44.644130	101.868495	11.211523	0.369345	23.5603	12.7074	11.42
4	49.712859	9.652075	28.317406	40.060784	108.168725	7.918501	0.543360	35.4940	15.9546	8.87

Next steps: Generate code with data | New interactive sheet

```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 310 entries, 0 to 309
Data columns (total 14 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   Col1         310 non-null    float64
 1   Col2         310 non-null    float64
 2   Col3         310 non-null    float64
 3   Col4         310 non-null    float64
 4   Col5         310 non-null    float64
 5   Col6         310 non-null    float64
 6   Col7         310 non-null    float64
 7   Col8         310 non-null    float64
 8   Col9         310 non-null    float64
 9   Col10        310 non-null    float64
 10  Col11        310 non-null    float64
 11  Col12        310 non-null    float64
 12  Class_att    310 non-null    object
 13  Unnamed: 13  14 non-null     object
dtypes: float64(12), object(2)
memory usage: 34.0+ KB
```

```
data = data.drop(columns=['Unnamed: 13'])
```

```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 310 entries, 0 to 309
Data columns (total 13 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   Col1       310 non-null    float64
 1   Col2       310 non-null    float64
 2   Col3       310 non-null    float64
 3   Col4       310 non-null    float64
 4   Col5       310 non-null    float64
 5   Col6       310 non-null    float64
 6   Col7       310 non-null    float64
 7   Col8       310 non-null    float64
 8   Col9       310 non-null    float64
 9   Col10      310 non-null    float64
 10  Col11      310 non-null    float64
 11  Col12      310 non-null    float64
 12  Class_att  310 non-null    object
dtypes: float64(12), object(1)
memory usage: 31.6+ KB
```

```
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()
data['Class_att'] = le.fit_transform(data['Class_att'])
```

```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 310 entries, 0 to 309
Data columns (total 13 columns):
 #   Column     Non-Null Count  Dtype  
---  ------     --------------  -----  
 0   Col1       310 non-null    float64
 1   Col2       310 non-null    float64
 2   Col3       310 non-null    float64
 3   Col4       310 non-null    float64
 4   Col5       310 non-null    float64
 5   Col6       310 non-null    float64
 6   Col7       310 non-null    float64
 7   Col8       310 non-null    float64
 8   Col9       310 non-null    float64
 9   Col10      310 non-null    float64
 10  Col11      310 non-null    float64
 11  Col12      310 non-null    float64
 12  Class_att  310 non-null    int64  
dtypes: float64(12), int64(1)
memory usage: 31.6 KB
```

```
data.head()
print(le.classes_)
```

```
['Abnormal' 'Normal']
```

```
data['Class_att'].value_counts()
```

Class_att	count
0	210
1	100

dtype: int64

```
X = data.drop('Class_att', axis=1)
y = data['Class_att']
```

```
#Train Test Split
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
#Feature Scaling
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
```

```
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
#Classification Model
#Model:Logistic Regression
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

model = LogisticRegression()

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)

print("Classification Accuracy:", accuracy)
```

```
Classification Accuracy: 0.8548387096774194
```

```
#Model:Regression Model
X_reg = data.drop('Col1', axis=1)
y_reg = data['Col1']
```

```
X_train_r, X_test_r, y_train_r, y_test_r = train_test_split(
    X_reg, y_reg, test_size=0.2, random_state=42
)
```

```
#Train regression model
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

reg_model = LinearRegression()

reg_model.fit(X_train_r, y_train_r)

y_pred_r = reg_model.predict(X_test_r)

print("MSE:", mean_squared_error(y_test_r, y_pred_r))
print("R2 Score:", r2_score(y_test_r, y_pred_r))
```

```
MSE: 2.2996594905375867e-17
R2 Score: 1.0
```

```
for i in range(10):
    print("Actual Col1 =", y_test_r.iloc[i])
    print("Predicted Col1 =", y_pred_r[i])
    print()
```

```
Actual Col1 = 44.43070103
Predicted Col1 = 44.43070103142567
```

```
Actual Col1 = 36.68635286
Predicted Col1 = 36.686352861833704

Actual Col1 = 46.85578065
Predicted Col1 = 46.85578064913279

Actual Col1 = 74.37767772
Predicted Col1 = 74.37767771949106

Actual Col1 = 54.12492019
Predicted Col1 = 54.12492018983376

Actual Col1 = 77.69057712
Predicted Col1 = 77.69057711902201

Actual Col1 = 85.35231529
Predicted Col1 = 85.35231529016653

Actual Col1 = 81.05661087
Predicted Col1 = 81.0566108701413

Actual Col1 = 63.02630005
Predicted Col1 = 63.02630005878563

Actual Col1 = 48.332638
Predicted Col1 = 48.33263799959402
```

```
#Clustering
X_cluster = data.drop('Class_att', axis=1)
```

```
scaler = StandardScaler()
X_cluster = scaler.fit_transform(X_cluster)
```

```
#Apply KMeans
from sklearn.cluster import KMeans

kmeans = KMeans(n_clusters=2, random_state=42)

clusters = kmeans.fit_predict(X_cluster)
```

```
#Silhouette Score
from sklearn.metrics import silhouette_score

sil = silhouette_score(X_cluster, clusters)

print("Silhouette Score:", sil)
```

```
Silhouette Score: 0.1783813198305097
```

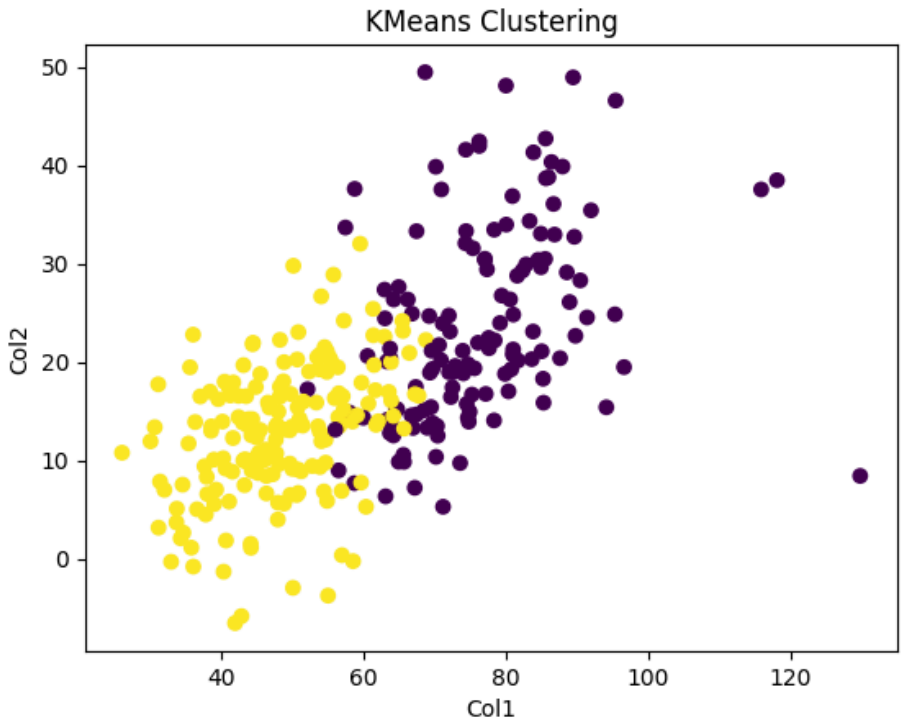
```
#Purity Score
from sklearn.metrics.cluster import contingency_matrix
import numpy as np

def purity_score(y_true, y_pred):
    matrix = contingency_matrix(y_true, y_pred)
    return np.sum(np.amax(matrix, axis=0)) / np.sum(matrix)
```

```
purity = purity_score(data['Class_att'], clusters)

print("Purity Score:", purity)
```

```
Purity Score: 0.6774193548387096
```

Inference: The clustering performance was evaluated using silhouette score and purity score. The silhouette score of 0.178 indicates weak separation between clusters. The purity score of 0.677 indicates that approximately 67.7 percent of the data points in the clusters correspond to the correct class labels. This suggests that clustering provides a moderate grouping of the spine condition data.

```
#Cluster Visualization
import matplotlib.pyplot as plt

plt.scatter(data['Col1'], data['Col2'], c=clusters)
plt.xlabel("Col1")
plt.ylabel("Col2")
plt.title("KMeans Clustering")
plt.show()
```



Inference: In this experiment, supervised and unsupervised machine learning techniques were applied to the Lower Back Pain Symptoms dataset containing 310 observations and 13 attributes.

For supervised learning, Logistic Regression was used for classification to predict whether a patient has a normal or abnormal spine condition based on spine measurement features. The model successfully classified the data using the attribute Class_att as the target variable.

For regression analysis, Linear Regression was applied to predict a continuous variable (Col1) using the remaining attributes. The regression model achieved a very low Mean Squared Error and a high R^2 score, indicating that the predicted values closely match the actual values.

For unsupervised learning, KMeans clustering was used to group the data into two clusters based on similarities in spine measurements. The quality of clustering was evaluated using Silhouette Score and Purity Score. The Silhouette Score of approximately 0.178 indicates that the clusters are not strongly separated,

while the Purity Score of about 0.677 shows that around 67 percent of the clustered data points correspond to the correct class labels.

Overall, the supervised learning models performed well in predicting spine conditions and numerical values, while the clustering approach provided moderate grouping of the dataset based on feature similarity.